



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2022/2023

Computação Física
Relatório: Trabalho Prático 3

Turma: 22D	Grupo: 6
Nome: Miguel Alcobia	Número: 50746
Nome: Fábio Pestana	Número: 50756
Nome: Rafael Dias	Número: 50773
Professor: Jorge Pais	

Data: 6 de Junho de 2023

Índice

Lista de figuras	II
Abreviaturas e símbolos.....	III
Objetivos	1
Parte 1	2
Enunciado:	2
1. 1 - Explicação do diagrama de atividades para interação com o display alfanumérico	2
1. 2 - Explicação do diagrama de atividades para interação com o display alfanumérico	2
1. 3 - Explicação do diagrama de atividades para interação com o sensor LPS22HB	3
1. 4 - Explicação do diagrama de atividades para interação com o sensor HS300X	4
1. 5 - Explicação da implementação do diagrama de atividades para interação com o display alfanumérico	5
1. 6 - Explicação da implementação do diagrama de atividades para interação com o sensor LPS22HB	6
1. 7 - Explicação da implementação do diagrama de atividades para interação com o sensor HS300X	7
Parte 2	8
Enunciado:	8
1. 1 - Explicação da abordagem da comunicação série entre o Arduino e o computador	8
1. 2 - Interface gráfica da estação meteorológica	10
Bibliografia	11
Código Comentado	12
Parte 1:	12
Parte 2:	20

Lista de figuras

Imagem 1 - Esquema de ligações entre o Arduino e o display 2

Imagem 2 - Diagrama UML de inicialização e de apresentação dos dados do display respetivamente3

Imagem 3 - Diagrama UML do sensor LPS22HB4

Imagem 4 - Diagrama UML do sensor HS300X.....4

Imagem 5 - Diagrama UML do autómato para o envio da string formatada para Python..... 10

Imagem 6 - Interface em Python 10

Abreviaturas e símbolos

Lista de abreviaturas

I ² C	Inter-Integrated Circuit
UML	Unified Modeling Language

Objetivos

Neste trabalho, os alunos tinham como tarefa criar uma pequena estação meteorológica.

Para realizar este trabalho, os alunos tiveram de estudar a arquitetura do Arduino Nano 33 BLE Sense Rev2, dos seus respetivos: sensores de temperatura (HS300X) e de pressão (LPS22HB); e do display (JDH1802) com o qual tinham de estabelecer uma comunicação com o Arduino através do protocolo I²C. No final, desta primeira parte, desenvolveram-se autómatos não bloqueantes para um bom funcionamento do sistema.

Na segunda parte do trabalho, os alunos tinham de realizar uma comunicação série entre o Arduino e o computador. Para este fim, foi utilizada a linguagem Python que também foi usada para a realização da interface gráfica que apresentaria os valores desejados.

Parte 1

Enunciado:

“Projete uma estação meteorológica, com informação da temperatura, pressão atmosférica, altitude e humidade relativa, mostradas num display alfanumérico. Para a medição das grandezas físicas, utilize os sensores presentes na placa do Arduino Nano 33 BLE Sense Rev2. A informação da estação meteorológica deve ser mostrada, a cada 10 segundos, no display de 2 linhas de 16 caracteres com interface I2C ”

- Carvalho, Carlos; Niehus, Manfred & Pais, Jorge. (2023). *Computação Física 3.º Trabalho Prático*

1. 1 - Explicação do diagrama de atividades para interação com o display alfanumérico

A imagem abaixo ilustra as ligações efetuadas entre o microcontrolador do Arduino e o display JDH1802, tomando o devido cuidado com pinos analógicos A4 e A5, pelos quais passariam os sinais digitais Serial Data (SDA) e Serial Clock (SCL). Estes sinais são a base do funcionamento do protocolo I²C.

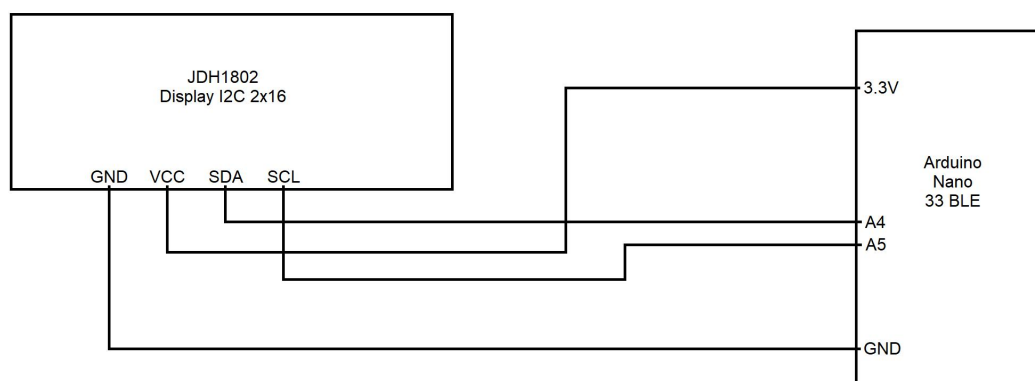


Imagem 1 - Esquema de ligações entre o Arduino e o display

1. 2 - Explicação do diagrama de atividades para interação com o display alfanumérico

Todos os diagramas de atividades foram desenhados seguindo a linguagem UML (*Unified Modeling Language*). Em todos os componentes (sensor de pressão, sensor de temperatura e

humidade e o display) a primeira versão do código foi elaborada sem autómatos; portanto a idealização dos diagramas de atividades teve como objetivos otimizar o funcionamento do sistema e a melhor compreensão do código Arduino.

Para a integração do display desenhou-se dois diagramas: um para a sua inicialização e outro para a apresentação dos dados desejados no LCD, correspondem ao métodos **displayI()** e **Display_Show()**, respetivamente.

Para a sua inicializção criou-se quatro estados. Cada um corresponde a umas das instruções que o fabricante indica para realizar este processo. Declarou-se uma variável **t0** para que em conjunto do método **millis()** (relógio do Arduino) se marque os tempos que o fabricante indica para cada instrução.

Para a apresentação dos dados criou-se dois estados: um estado de espera (**Idle**) e outro para posicionar o cursor e escrever os dados desejados (**Display**). Assim como no caso anterior, neste também se criou uma variável para trabalhar com o tempo do Arduino, para respeitar o tempo de 10 segundos do enunciado.

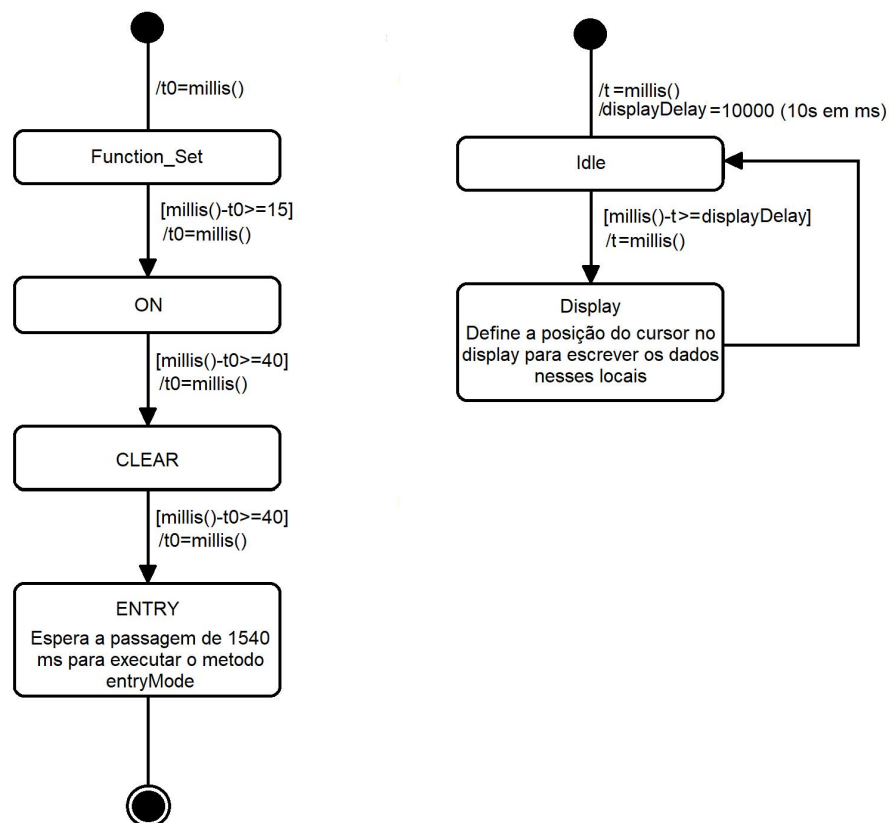


Imagem 2 - Diagrama UML de inicialização e de apresentação dos dados do display respetivamente

1. 3 - Explicação do diagrama de atividades para interação com o sensor LPS22HB

Para a integração do sensor de pressão LPS22HB desenhou-se um diagrama com três estados.

Cada um corresponde a umas das instruções que o fabricante indica para realizar o modo “OneShot”.

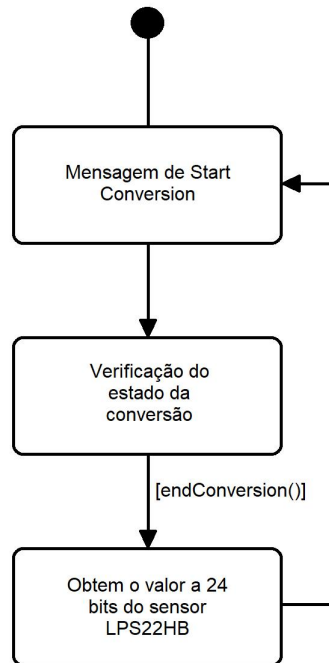


Imagem 3 - Diagrama UML do sensor LPS22HB

1. 4 - Explicação do diagrama de atividades para interação com o sensor HS300X

Para a integração do sensor de temperatura e humidade (HS300X) desenhou-se um diagrama com dois estados. O primeiro representa o envio da mensagem “*start conversion*” e o segundo representa o estado final da conversão, onde se faz o pedido da medição dos valores desejados e se efetua a sua respetiva conversão para graus Celsius e percentagem de humidade.

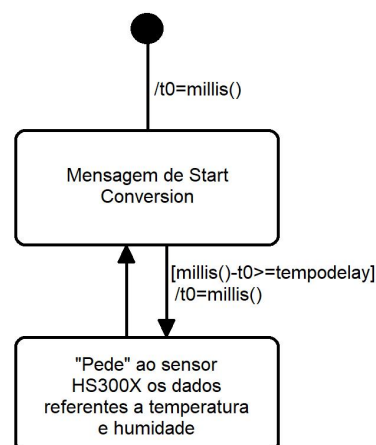


Imagem 4 - Diagrama UML do sensor HS300X

1.5 - Explicação da implementação do diagrama de atividades para interação com o display alfanumérico

```
void displayI()
{
    static int state=Function_Set;
    static unsigned long t0=millis();
    switch (state) {
        case Function_Set:
            if(millis()-t0>=15)
            {
                functionSet();
                t0=millis();
                state=ON;
            }
            break;
        case ON:
            if(millis()-t0>=40)
            {
                displayOn();
                t0=millis();
                state=CLEAR;
            }
            break;
        case CLEAR:
            if(millis()-t0>=40)
            {
                displayClear();
                t0=millis();
                state=ENTRY;
            }
            break;
        case ENTRY:
            if(millis()-t0>=1540)
            {
                entryMode();
            }
            break;
    }
}
```

Para a implementação do diagrama da inicialização do display criaram-se os já referidos quatro estados. No estado **Function_Set**, chama-se o método que corresponde à instrução desejada, logo após do tempo de espera indicados pelo fabricante . O mesmo acontece para o resto dos casos, com a diferença que o estado **ENTRY** não muda para outro estado, pois a inicialização do display deverá ocorrer apenas uma vez.

```

void DisplayShow(float temperatura, float humidade,
float pressao, float altitude)
{
    static int state=Idle;
    static unsigned long t=millis();
    int displayDelay = 10000; //10 segundos
    switch (state) {
        case Idle:
            if(millis()-t>displayDelay)
            {
                t=millis();
                state=Display;
            }
            break;
        case Display:
            setCursor(0, 2);
            printString(String(humidade));
            setCursor(0, 10);
            printString(String(temperatura));
            setCursor(1, 2);
            printString(String(pressao, 1));
            setCursor(1, 10);
            printString(String(altitude));
            state=Idle;
            break;
    }
}

```

Para a implementação do diagrama da exibição dos dados no display criaram-se os já referidos dois estados. No estado **Idle**, é feita uma espera de 10 segundos, tal como pedido no enunciado e só então se passa para o próximo estado onde ocorre a exibição dos valores . No estado **Display** posiciona-se o cursor e, em seguida, é feito o print do valor no display. Este processo é repetido para os restantes valores desejados e ao estarem todos exibidos, volta-se ao estado **Idle** onde esperaremos mais 10 segundos para uma atualização nos valores.

1. 6 - Explicação da implementação do diagrama de atividades para interação com o sensor LPS22HB

```

void LPS22HB(float *pressaohPa)
{
    static int state=SC;
    switch (state) {
        case SC:
            startConversionP();
            state=GoOrNo;
            break;
        case GoOrNo:
            if (endConversion())
            {
                state=ReadP;
            }
            break;
        case ReadP:
            lerPressao(pressaohPa);
            state=SC;
            break;
    }
}

```

Para a implementação do diagrama do sensor da pressão criaram-se os já referidos três estados. No estado **SC**, chama-se o método que corresponde à respetiva instrução do método “OneShot”. O mesmo ocorre para todos os estados. No **GoOrNo**, é feita a verificação do estado da conversão pelo método **endConversion()**, se este retornar o valor true significa que é permitida a passagem para o próximo estado (**ReadP**), no qual será lida a pressão.

1. 7 - Explicação da implementação do diagrama de atividades para interação com o sensor HS300X

```
void HS300X(float *temperatura, float *humidade)
{
    static int state=StartC;
    static unsigned long t0=millis();
    switch (state) {
        case StartC:
            startConversion();
            if(millis()-t0>=tempodelay)
            {
                t0=millis();
                state=EndC;
            }
            break;
        case EndC:
            pedidoMedicao(temperatura, humidade);
            state=StartC;
            break;
    }
}
```

Para a implementação do diagrama do sensor da temperatura e humidade criaram-se os já referidos dois estados. No estado **StartC**, chama-se o método **startConversion()** e após 34ms (tempo indicado pelo fabricante) mudamos para o segundo estado. No **EndC**, é efetuado o pedido de medição da temperatura e da humidade, onde se realiza a sua leitura e respetiva conversão.

Parte 2

Enunciado:

“Desenhe um painel de afixação outdoor, que inclui a estação meteorológica, com capacidade de refrescar informação a cada 10 segundos, acerca da temperatura, pressão atmosférica, altitude e humidade relativa. Para este efeito, o painel outdoor é um monitor dum PC, onde a informação deve ser visualizada em modo texto, a partir duma aplicação desenvolvida em Python. Como opção de melhoramento, pode ser implementada uma interface gráfica em Python, onde a informação será mostrada de uma forma imaginativa e apelativa. Esta opção de melhoramento será valorizada com um acréscimo máximo de 2 valores na nota final do trabalho.”

- Carvalho, Carlos; Niehus, Manfred & Pais, Jorge. (2023). *Computação Física 3.º Trabalho Prático*

1. 1 - Explicação da abordagem da comunicação série entre o Arduino e o computador

Para estabelecer a comunicação entre o Arduino e o computador, foi utilizada a linguagem Python.

Para a realização desta comunicação elaborou-se o seguinte código:

```
### COMUNICAÇÃO SERIAL ###

# Lista de portas serial encontradas
ports = serial.tools.list_ports.comports()

# Cria uma instância da classe Serial
serialInst = serial.Serial()

# Lista para guardar as portas encontradas
portList = []

# Percorre todas as portas e coloca-as na Lista.
for porta in ports:
    portList.append(str(porta))
    print(str(porta))

# Pede ao utilizador que insira o numero da porta que está a usar
val = input("Escolha a porta: COM")

# Percorre um lista de portas e verifica se alguma corresponde com a
escolha do utilizador
for x in range(0, len(portList)):
    if portList[x].startswith("COM" + str(val)):
        portVar = "COM" + str(val)
        print(portVar)
        print(portList[x])

# Continua na próxima página
```

```

# Define o baudrate do canal Serial para 9600 bps (bits por segundo)
serialInst.baudrate = 9600

# Atribui a porta selecionada à instância serial
serialInst.port = portVar

# Abre a conexão serial
serialInst.open()

# Chama a função para criar a janela e começar a atualizar os dados
criar_janela()

while True:
    if serialInst.in_waiting:
        # Quando houver dados disponíveis na conexão serial, atualiza a janela

        packet = serialInst.readline()
        criar_janela().atualizar_dados()
        schedule.run_pending()
        time.sleep(1)

```

O código ainda tem os seguintes *imports*:

```

import serial.tools.list_ports
import serial

```

Estes *imports*, importam a biblioteca pySerial, o primeiro especificamente importa o módulo **tools.list_ports**, que pode ser executado para obtermos uma lista de portas disponíveis.

O comando **serial.tools.list_ports.comports()** retorna a lista de portas que temos à nossa disposição, sem nenhuma ordem específica.

Na variável **serialInst** é criada um instância da classe **Serial**, que servirá para abrir a porta escolhida e fazer as devidas configurações.

O array portList servirá para guardar as portas encontradas no primeiro ciclo *for* .

O utilizador introduz o valor da porta que deseja utilizar e o segundo ciclo *for* verifica se esta existe, caso exista imprime o seu valor e então procede-se à sua configuração com o valor de *baurate* (9600).

No final procede-se à leitura de valores vindos do Arduino.

Como os valores estavam a ser lidos como uma única *string*, então procedemos a um método simples para filtrar os dados desejados. Este filtro consiste em limitar os dados da temperatura com T's, os da humidade com H's , os da altitude com A's e os da pressão com P's.

Para o envio desta string formatada elaborou-se um autómato **Python** no código Arduino.

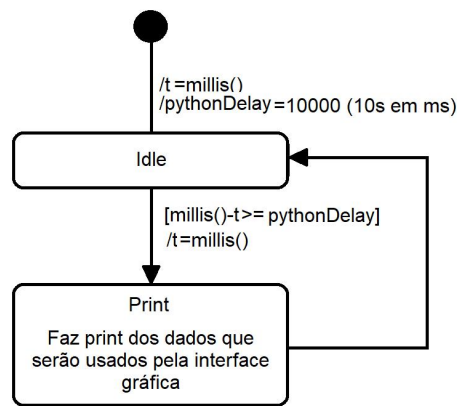


Imagem 5 - Diagrama UML do autômato para o envio da string formatada para Python

1.2 - Interface gráfica da estação meteorológica

Para a interface planeou-se algo bastante simples: uma janela que imprimia os valores desejados. Para a temperatura em específico, planeou-se uma barra que simularia um termómetro, com uma escala de 0 a 100 graus Celsius. A barra seria mais ou menos preenchida, consoante o valor recebido da temperatura.

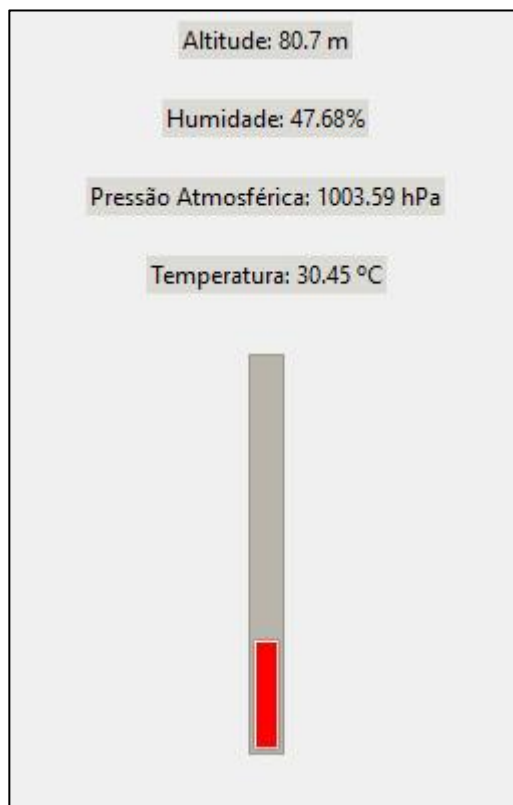


Imagem 6 - Interface em Python

Bibliografia

Consulta:

Python:

TinkerAssist (2021). *Python Tutorial - How to Read Data from Arduino via Serial Port*. YouTube. <https://www.youtube.com/watch?v=AHr94RtMj1A>

Liechti Chris (2020). *Tools — pySerial 3.4 documentation*. Read the Docs. <https://pyserial.readthedocs.io/en/latest/tools.html>

Google (n.d.). *Google Python Style Guide - google/styleguide/pyguide*. GitHub. <https://github.com/google/styleguide/blob/gh-pages/pyguide.md#38-comments-and-docstrings>

Moodle:

Pais, Jorge. (2022 - 2023). *Computação Física*

Carvalho, Carlos (2023). *Computação Física*

Desenhos:

MS Paint

Draw.io - <https://app.diagrams.net/>

Código Comentado

Parte 1:

```
//ISEL- LEIM - Grupo 06 - Computação Física - TP3
//Coding UTF-08

//include's
#include <Wire.h>
#include <math.h>

//define's
#define enderecoI2C_HS300X 0x44
#define tempodelay 34 // 34 ms é o tempo de aquisição para o sensor obter uma nova temperatura a 14 bits

#define ENDERECO 0x3E
#define RS 0x40
#define CO 0x80

#define LCD_FUNCTIONSET 0x20
#define LCD_2LINHAS 0x08
#define LCD_5X10 0x04

#define LCD_DISPLAYONOFF 0x08
#define LCD_DISPLAYON 0x04
#define LCD_CURSORON 0x02
#define LCD_BLINKON 0x01

#define CO 0x80

#define LCD_DISPALYCLEAR 0x01

#define LCD_ENTRYMODE 0x04
#define LCD_I 0x02

#define enderecoI2C_LPS22B 0x5C
#define enderecoRegistoCTRL_REG2 0x11
#define LPS22HB_PXL 0x28 // XLSB
#define LPS22HB_PL 0x29 // LSB
#define LPS22HB_PH 0x2A // MSB

//define's dos estados dos autómatos
#define StartC 0
#define EndC 1

#define Idle 0
#define Display 1

#define Idle 0
#define Print 1

#define Function_Set 0
#define ON 1
#define CLEAR 2
#define ENTRY 3

#define SC 0
#define GoOrNo 1
#define ReadP 2

//variáveis
word humidade14bit, temperatura14bit;
float humidadeP, temperaturaGraus;
float altitude, pressaohPa;
byte valor8bits;
byte valor24bits;

byte Cursor;

///// HS300X
```



```

/**
 * Este método lê os dados de humidade e temperatura do sensor.
 * Extrai os bits de interesse para a humidade e temperatura
 * @param temperatura saída atualizada com os dados do sensor
 * @param humidade saída atualizada com os dados do sensor
 */
void lerHumidadeTemperaturaBinario14bit(word *temperatura, word *humidade) {
    if(Wire1.requestFrom(enderecoI2C_HS300X, 4) == 4) {
        word H_high = Wire1.read() & 0xFF;
        word H_low = Wire1.read() & 0xFF;
        word T_high = Wire1.read() & 0xFF;
        word T_low = Wire1.read() & 0xFF;

        //14 bits 11 1111 1111 1111-> 0x3FFF
        *humidade = ((H_high << 8) | H_low) & 0x3FFF;
        *temperatura = ((T_high << 8) | T_low) >> 2 & 0x3FFF;
    }
}

/**
 * Este método converte o valor binário da humidade para um valor em percentagem
 * @param humidade14bits valor binário da humidade a 14 bits
 * @param humidadePercentagem saída com os dados humidade14bits convertidos em percentagem
 */
void converterHumidade(word humidade14bits, float *humidadePercentagem){
    *humidadePercentagem= (float)humidade14bits / (float) 0x3FFF * 100;
}

/**
 * Este método converte o valor binário da temperatura para um valor em graus Celsius
 * @param temperatura14bits valor binário da temperatura a 14 bits
 * @param temperaturaGraus saída com os dados temperatura14bits convertidos em graus Celsius
 */
void converterTemperatura(word temperatura14bits, float *temperaturaGraus){
    *temperaturaGraus= (float)temperatura14bits / (float)0x3FFF * 165.0- 40.0;
}

/**
 * Este método envia a mensagem start conversion para o sensor HS300X
 */
void startConversion(){
    Wire1.beginTransaction(enderecoI2C_HS300X); // iniciar mensagem
    Wire1.write(enderecoI2C_HS300X); // ativar sinal de escrita
    Wire1.endTransmission(); // terminar mensagem
}

/**
 * Este método usa a função lerHumidadeTemperaturaBinario14bit() para atualizar os valores da humidade e
da temperatura
 * Para depois os converter usando as funções converterTemperatura() e converterHumidade()
 * @param temperatura saída com os dados atualizados da temperatura
 * @param humidade saída com os dados atualizados da humidade
 */
void pedidoMedicao(float *temperatura, float *humidade)
{
    lerHumidadeTemperaturaBinario14bit(&temperatura14bit, &humidade14bit);
    converterTemperatura(temperatura14bit, temperatura);
    converterHumidade(humidade14bit, humidade);
}

/**
 * Este método é um autómato com o funcionamento do sensor HS300X
 * No estado inicial, StartC, usa-se a função startConversion() e espera-se 34 ms
 * No segundo estado, EndC, usa-se o método pedidoMedicao()
 * para atualizar os valores da temperatura e humidade
 * @param temperatura saída com os dados atualizados da temperatura
 * @param humidade saída com os dados atualizados da humidade
 */
void HS300X(float *temperatura, float *humidade)
{
    static int state=StartC;
    static unsigned long t0=millis();
    switch (state) {
        case StartC:
            startConversion();

```

```

        if(millis()-t0>=tempodelay)
        {
            t0=millis();
            state=EndC;
        }
        break;
        case EndC:
            pedidoMedicao(temperatura, humidade);
            state=StartC;
            break;
    }
}

void setup() {
    Serial.begin(9600); // inicializa a comunicação serial
    Wire1.begin(); // inicializa a comunicação I2C
    Wire.begin();
    displayI();
    //Display_Iniciar();
    setCursor(0, 0);
    printString("H: ");
    setCursor(0, 8);
    printString("T: ");
    setCursor(1, 0);
    printString("P: ");
    setCursor(1, 8);
    printString("A: ");
}

void loop() {

    HS300X(&temperaturaGraus, &humidadeP);
    LPS22HB(&pressaohPa);
    converterPressao2Metros(pressaohPa, &altitude);

    // exibe os valores de humidade e temperatura na porta serial
    DisplayShow(temperaturaGraus, humidadeP, pressaohPa, altitude);

    Python();
}

///// Python

/**
 * Este método é um autómato para fazer Serial.println() dos valores na consola, de 10 em 10 segundos.
 * Estes valores seguem uma sequência e estão entre letras diferentes para facilitar a filtração dos
 * dados pelo Python
 */
void Python()
{
    static int state=Idle;
    static unsigned long t=millis();
    int pythonDelay = 10000; //10 segundos
    switch (state) {
        case Idle:
            if(millis()-t>pythonDelay)
            {
                t=millis();
                state=Print;
            }
            break;
        case Print:
            Serial.println(String("T") +temperaturaGraus+String("T")+ String("H")+humidadeP + String("H") +
String("P")+pressaohPa +String("P") +String("A") +altitude+ String("A"));
            state=Idle;
            break;
    }
}

///// Display

```

```

/**
 * Este método é um autômato para colocar/atualizar valores no Display I2C, de 10 em 10 segundos.
 * @param temperatura
 * @param humidade
 * @param pressao
 * @param altitude
 */
void DisplayShow(float temperatura, float humidade, float pressao, float altitude)
{
    static int state=Idle;
    static unsigned long t=millis();
    int displayDelay = 10000; //10 segundos
    switch (state) {
        case Idle:
            if(millis()-t>displayDelay)
            {
                t=millis();
                state=Display;
            }
            break;
        case Display:
            setCursor(0, 2);
            printString(String(humidade));
            setCursor(0, 10);
            printString(String(temperatura));
            setCursor(1, 2);
            printString(String(pressao, 1));
            setCursor(1, 10);
            printString(String(altitude));
            state=Idle;
            break;
    }
}

/**
 * Código fornecido pelas folhas do Professor
 * @param oitoBits escreve 1 byte no display
 */
void escreverDados8(byte oitoBits)
{
    Wire.beginTransmission(ENDERECO);
    Wire.write(RS);
    Wire.write(oitoBits);
    Wire.endTransmission();
    delayMicroseconds(45); // tempo > 44us para comando fazer efeito
}

/**
 * Este método é um autômato baseado na sequência de instruções do fabricante do display.
 */
void displayI()
{
    static int state=Function_Set;
    static unsigned long t0=millis();
    switch (state) {
        case Function_Set:
            if(millis()-t0>=15)
            {
                functionSet();
                t0=millis();
                state=ON;
            }
            break;
        case ON:
            if(millis()-t0>=40)
            {
                displayOn();
                t0=millis();
                state=CLEAR;
            }
            break;
        case CLEAR:
            if(millis()-t0>=40)
            {
                displayClear();
                t0=millis();
                state=ENTRY;
            }
    }
}

```

```

        }
        break;
    case ENTRY:
        if(millis()-t0>=1540)
        {
            entryMode();
        }
        break;
    }
}

/**
 * Este método é baseado na sequência de instruções do fabricante do display
 * Versão de inicialização do display com delay()'s
 */
void Display_Iniciar()
{
    delay(15);
    functionSet();
    delayMicroseconds(40);
    displayOn();
    delayMicroseconds(40);
    displayClear();
    delayMicroseconds(1540);
    entryMode();
}

/**
 * Este método escreve dados no display (CO e oitoBits)
 * @param oitoBits escreve 1 byte no display
 */
void escreverComando8(byte oitoBits)
{
    Wire.beginTransaction(ENDereco);
    Wire.write(CO);
    Wire.write(oitoBits);
    Wire.endTransmission();
}

/**
 * Este método usa a função escreverComando8() para ligar o display
 */
void displayOn()
{
    escreverComando8(LCD_DISPLAYONOFF|LCD_DISPLAYON|LCD_CURSORON|LCD_BLINKON);
}

/**
 * Este método usa a função escreverComando8() para desligar o display
 */
void displayOff()
{
    escreverComando8(LCD_DISPLAYONOFF);
}

/**
 * Este método usa a função escreverComando8() para atribuir funcionalidades ao display
 */
void functionSet()
{
    escreverComando8(LCD_FUNCTIONSET|LCD_2LINHAS|LCD_5X10);
}

/**
 * Este método usa a função escreverComando8() para atribuir funcionalidades ao cursor do display
 */
void entryMode()
{
    escreverComando8(LCD_ENTRYMODE|LCD_I);
}

/**
 * Este método usa a função escreverComando8() para "limpar" o display
 */
void displayClear()
{
    escreverComando8(LCD_DISPALYCLEAR);
}

```

```

}

/**
 * Este método usa a função escreverComando8() para colocar o cursor numa posição do display
 * @param linha byte que representa uma linha do display
 * @param coluna byte que representa uma coluna do display
 */
void setCursor(byte linha, byte coluna)
{
    Cursor = 0x80 | linha<<6 | coluna & 0x3F; //A6 -> linha; A5-0 -> coluna
    escreverComando8(Cursor);
}

/**
 * Este método usa a função escreverDados8() para escrever o char c no display
 * @param c char que será escrito no display
 */
void printchar(char c)
{
    escreverDados8(c);
}

/**
 * Este método usa a função printchar() para escrever o conteúdo da String (char a char) no display
 * * até encontrar um char = '\0'
 * @param s String que será escrita no display
 */
void printString (String s)
{
    int i= 0;
    while(s[i]!='\0'){
        printchar(s[i]);
        i++;
    }
}

///// LPS22HB

/**
 * Este método é um autômato com o funcionamento do sensor LPS22HB
 * No estado inicial, SC, usa-se a função startConversionP()
 * No segundo estado, GoOrNo, espera-se que o metodo endConversion() retorne true
 * No ultimo estado , ReadP, usa-se o método lerPressao() para atualizar o valor da pressao
 * @param pressaohPa saída com os dados atualizados da pressao
 */
void LPS22HB(float *pressaohPa)
{
    static int state=SC;
    switch (state) {
        case SC:
            startConversionP();
            state=GoOrNo;
            break;
        case GoOrNo:
            if (endConversion())
            {
                state=ReadP;
            }
            break;
        case ReadP:
            lerPressao(pressaohPa);
            state=SC;
            break;
    }
}

/**
 * Este método é a versao anterior ao LPS22HB()
 * Substituida por um autômato devido a utilização do while()
 * @param pressaohPa saída com os dados atualizados da pressao
 */
void oneShot(float *pressaohPa){
    startConversionP();
    while (!endConversion());
    lerPressao(pressaohPa);
}

```

```

/**
 * Este método envia a mensagem start conversion para o sensor LPS22HB
 */
void startConversionP() {
    byte valor;
    lerRegisto8bit(enderecoI2C_LPS22B, enderecoRegistoCTRL_REG2, &valor); // ler registo CTRL_REG2 de 8 bits
    via I2C
    escreverRegisto8bits(enderecoI2C_LPS22B, enderecoRegistoCTRL_REG2, valor | 0x01); // afetar com o valor
    lógico 1 o bit 0 do registo CTRL_REG2 e escrever o novo valor no CTRL_REG2
}

/**
 * Este método indica quando o sensor acaba a aquisição da pressão
 * @return true quando obter o seu valor
 * @return false caso contrário
 */
bool endConversion() {
    byte registoValor;
    lerRegisto8bit(enderecoI2C_LPS22B, enderecoRegistoCTRL_REG2, &registoValor); // Solicitar a leitura de 1
    byte do registo
    if(registoValor==0x10){
        return true;
    }else {
        return false;
    }
}

/**
 * Código fornecido pelas folhas do Professor
 * @param enderecoI2C
 * @param enderecoRegisto
 * @param valor8bits
 * @return true caso execute a leitura do byte
 * @return false caso não execute
 */
bool lerRegisto8bit(byte enderecoI2C, byte enderecoRegisto, byte *valor8bits) {
    Wire1.beginTransaction(enderecoI2C);
    Wire1.write(enderecoRegisto);
    Wire1.endTransmission();
    if (Wire1.requestFrom(enderecoI2C, 1)==1){
        *valor8bits = Wire1.read();
        return true;
    }
    return false;
}

/**
 * Este método permite a leitura a 24 bits do sensor LPS22HB
 * @param enderecoI2C
 * @param enderecoRegisto
 * @param valor24bits
 * @return true caso execute a leitura
 * @return false caso não execute
 */
bool lerRegisto24bit(byte enderecoI2C, byte enderecoRegisto, byte *valor24bits) {
    Wire1.beginTransaction(enderecoI2C);
    Wire1.write(enderecoRegisto);
    Wire1.endTransmission();

    if (Wire1.requestFrom(enderecoI2C, 3) == 3 ){

        byte MSB = Wire1.read();
        byte LSB = Wire1.read();
        byte XLSB = Wire1.read();

        *valor24bits = (MSB<< 16) | (LSB << 8) | XLSB;

        return true;
    }
    return false;
}

/**
 * Este método permite escrever 1 byte no sensor LPS22HB
 * @param enderecoI2C
 * @param enderecoRegisto
 * @param valor8bits

```

```

    */
void escreverRegisto8bits(byte enderecoI2C, byte enderecoRegisto, byte valor8bits) {
    Wire1.beginTransaction(enderecoI2C);
    Wire1.write(enderecoRegisto);
    Wire1.write(valor8bits);
    Wire1.endTransmission();
}

/**
 * Este método obtém o valor binario da pressão e converte em hPa
 * @param pressaoHPa valor float da pressao
 */
void lerPressao(float *pressaoHPa) {
    byte pressaoXL, pressaoL, pressaoH;

    if (lerRegisto24bit(enderecoI2C_LPS22B, LPS22HB_PXL, &pressaoXL) &&
        lerRegisto8bit(enderecoI2C_LPS22B, LPS22HB_PL, &pressaoL) &&
        lerRegisto8bit(enderecoI2C_LPS22B, LPS22HB_PH, &pressaoH)) {
        // Combinar os valores lidos para obter a pressão a 24 bits
        *pressaoHPa = ((word)pressaoH << 16 | (word)pressaoL << 8 | (word)pressaoXL)/4096.0;
    } else {
        *pressaoHPa = 0.0; // Valor padrão de pressão em caso de falha
    }
}

/**
 * Este método utiliza o valor da pressao para calcular o valor da altitude,
 * comparando a pressao atual com a do nivel das águas do mar
 * @param pressaoHPa valor float da pressao
 * @param altitude saída com os dados da altitude em metros
 */
void converterPressao2Metros(float pressaoHPa, float*altitude) {
    float pressaoNivelMarHPa = 1013.25;
    *altitude= 44300.0*(1-pow((pressaoHPa/pressaoNivelMarHPa), (1/5.255)));
}

```

Parte 2:

```
"""
ISEL- LEIM - Grupo 06 - Computação Física - TP3
Coding UTF-8
"""

# Imports das bibliotecas usadas no programa

import serial.tools.list_ports
import serial
import tkinter as tk
from tkinter import ttk
import time
import schedule

### INTERFACE ###

def criar_janela():
    # Cria a janela principal

    janela = tk.Tk()

    # Cria uma "label" que vai sendo atualizada com o valor da altitude

    altitude_label = ttk.Label(janela, text="Altitude: 0 m")
    altitude_label.pack(pady=10)

    # Cria uma "label" que vai sendo atualizada com o valor da humidade

    humidade_label = ttk.Label(janela, text="Humidade: 0%")
    humidade_label.pack(pady=10)

    # Cria uma "label" que vai sendo atualizada com o valor da pressão

    pressao_label = ttk.Label(janela, text="Pressão Atmosférica: 0 hPa")
    pressao_label.pack(pady=10)

    # Cria uma "label" que vai sendo atualizada com o valor da temperatura

    temperatura_label = ttk.Label(janela, text="Temperatura: 0 °C")
    temperatura_label.pack(pady=10)

    # Configuração e criação de uma "progressbar" colocada na horizontal, que serve como termómetro

    janela.grid()
    s = ttk.Style()
    s.theme_use('clam')
    s.configure("red.Horizontal.TProgressbar", foreground='red', background='red')
    progress_bar = ttk.Progressbar(janela, style="red.Horizontal.TProgressbar", orient="vertical",
    length=200,
                                mode="determinate")
    progress_bar.pack(pady=20)

    def atualizar_dados():
        # Receção dos valores de temperatura, humidade, pressão e altitude

        packet = serialInst.readline()
        dados = packet.decode('utf').rstrip('\n')
        temperatura = getTemperatura(dados)
        humidade = getHumidade(dados)
        pressao = getPressao(dados)
        altitude = getAltitude(dados)

        """
        # Verificar se o Python está a receber os valores enviados pelo arduino
        print(temperatura)
        print(humidade)
        print(pressao)
        print(altitude)
        """

        # Atualiza as "labels" com os dados

        altitude_label.config(text=f"Altitude: {float(altitude)} m")
```



```

humidade_label.config(text=f"Humidade: {float(humidade)}%")
pressao_label.config(text=f"Pressão Atmosférica: {float(pressao)} hPa")
temperatura_label.config(text=f"Temperatura: {float(temperatura)} °C")

# Atualiza a "progressbar"

progress_bar["value"] = float(temperatura)

# Agenda a próxima atualização após 1 segundo

janela.after(1000, atualizar_dados)

# Inicia a atualização dos dados
atualizar_dados()

def fechar_janela():
    janela.destroy()

janela.protocol("WM_DELETE_WINDOW", fechar_janela)

# Inicia a atualização dos dados

atualizar_dados()

janela.mainloop()

def getTemperatura(string):
    """
    Função para extrair o valor de temperatura da string recebida

    Args:
        string(str): string com o valor da temperatura

    Return:
        T(str): valor da temperatura
    """
    T = ''
    for i in range(len(string)):
        if string[i] == 'T':
            while string[i + 1] != 'T':
                T = T + string[i + 1]
                i += 1
            return T

def getHumidade(string):
    """
    Função para extrair o valor de humidade da string recebida

    Args:
        string(str): string com o valor da humidade

    Return:
        H(str): valor da humidade
    """
    H = ''
    for i in range(len(string)):
        if string[i] == 'H':
            while string[i + 1] != 'H':
                H = H + string[i + 1]
                i += 1
            return H

def getPressao(string):
    """
    Função para extrair o valor de pressão da string recebida

    Args:
        string(str): string com o valor da pressão

    Return:
        P(str): valor da pressão
    """
    P = ''

```

```

    for i in range(len(string)):
        if string[i] == 'P':
            while string[i + 1] != 'P':
                P = P + string[i + 1]
                i += 1
            return P

def getAltitude(string):
    """
        Função para extrair o valor de altitude da string recebida

        Arg:
            string(str): string com o valor da altitude

        Return:
            A(str): valor da altitude
    """
    A = ''
    for i in range(len(string)):
        if string[i] == 'A':
            while string[i + 1] != 'A':
                A = A + string[i + 1]
                i += 1
            return A

### COMUNICAÇÃO SERIAL ###

# Lista de portas serial encontradas
ports = serial.tools.list_ports.comports()

# Cria uma instância da classe Serial
serialInst = serial.Serial()

# Lista para guardar as portas encontradas
portList = []

# Percorre todas as portas e coloca-as na Lista.
for porta in ports:
    portList.append(str(porta))
    print(str(porta))

# Pede ao utilizador que insira o numero da porta que está a usar
val = input("Escolha a porta: COM")

# Percorre um lista de portas e verifica se alguma corresponde com a escolha do utilizador
for x in range(0, len(portList)):
    if portList[x].startswith("COM" + str(val)):
        portVar = "COM" + str(val)
        print(portVar)
        print(portList[x])

# Define o baudrate do canal Serial para 9600 bps (bits por segundo)
serialInst.baudrate = 9600

# Atribui a porta selecionada à instância serial
serialInst.port = portVar

# Abre a conexão serial
serialInst.open()

# Chama a função para criar a janela e começar a atualizar os dados
criar_janela()

```

```
while True:
    if serialInst.in_waiting:
        # Quando houver dados disponíveis na conexão serial, atualiza a janela

        packet = serialInst.readline()
        criar_janela().atualizar_dados()
        schedule.run_pending()
        time.sleep(1)
```